MCB 536: Tools for Computational Biology

Lecture 12:
Data analysis with
R and tidyverse

Bo Yuan

Postdoctoral Fellow, Fred Hutch

Goals for this week's classes

- Basic R syntax and logic
- Data analysis and visualization of tabular data with R and tidyverse



R and tidyverse

R is designed for statistical and mathematical computing.

tidyverse is a collection of packages for data analysis.

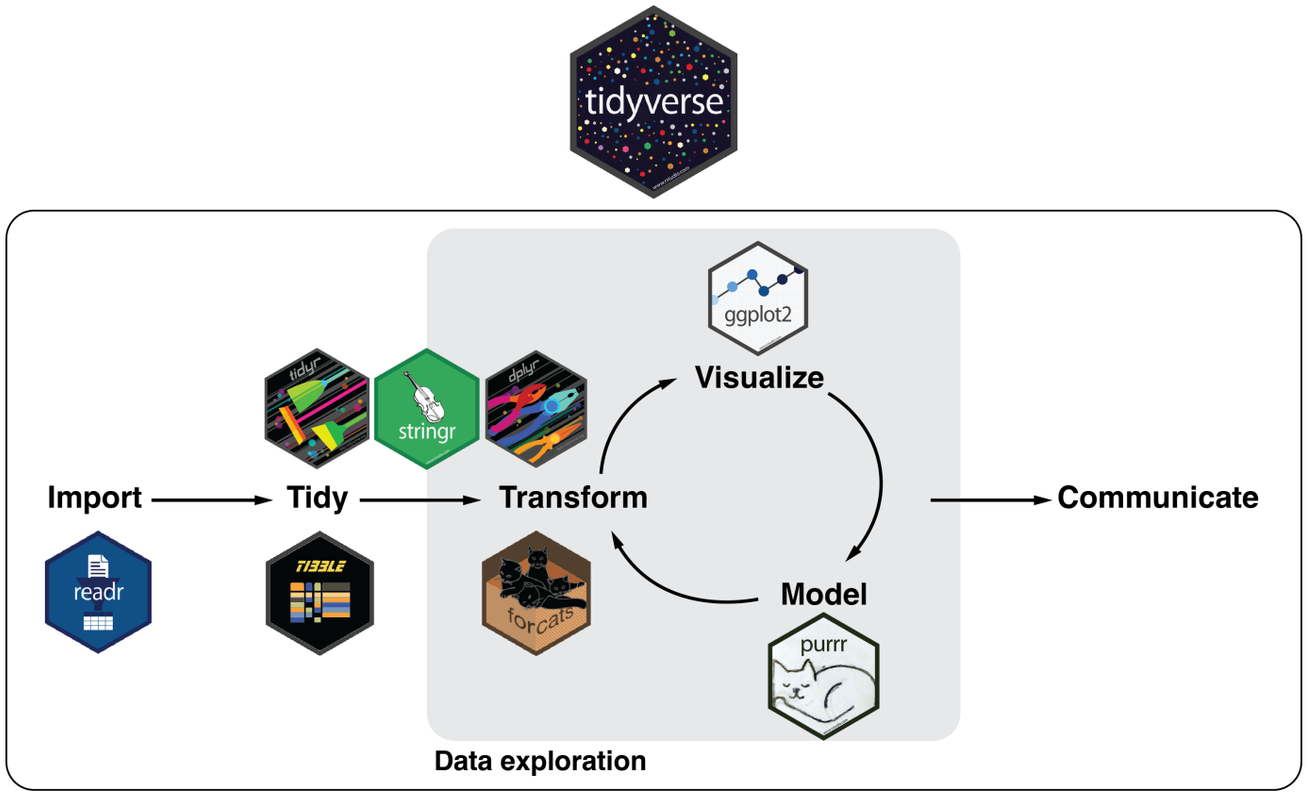


Figure adapted from R for Data Science, 2017

Install and load packages

Install a package

```
install.packages("tidyverse")
```

Load the package

```
library(tidyverse)
```

Import data

Read a CSV file

```
data <- read_csv("data.csv")
```

Read a tab-separated file

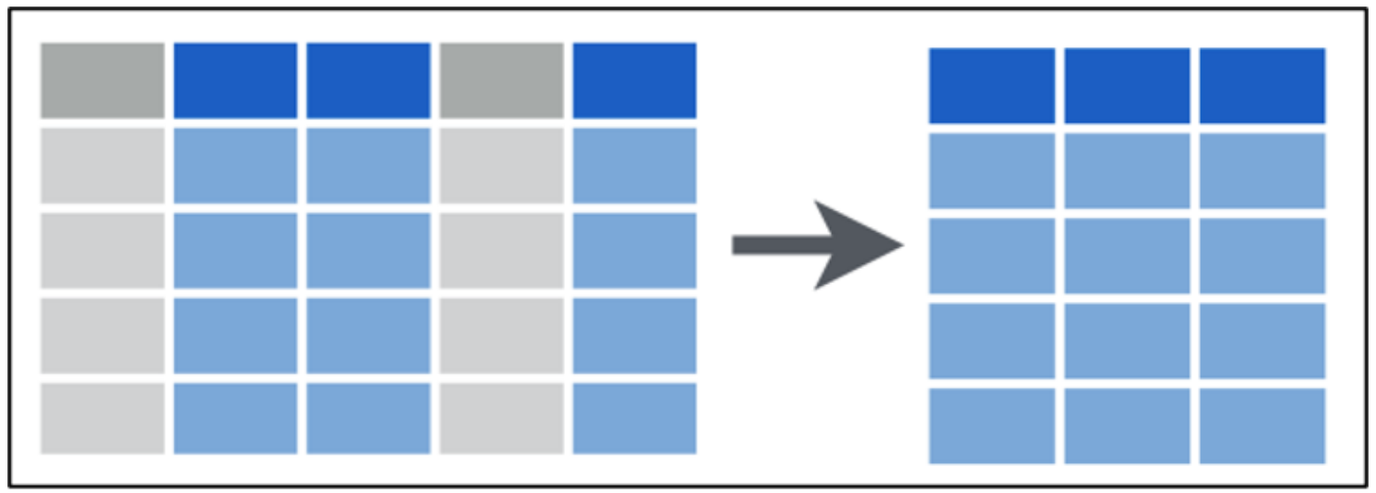
```
data <- read_tsv("data.tsv")
```

Select columns

tidyverse provides functions for tidying, manipulating, and transforming tabular data.

select( ) keeps columns in a tibble or data frame.

```
select(dataframe, column_name1, column_name2, ...)
```



Example: Keep age column from data

```
select(data, age)
```

Filter rows

filter( ) keeps rows that meet a condition.

```
filter(dataframe, logical_expression, ...)
```



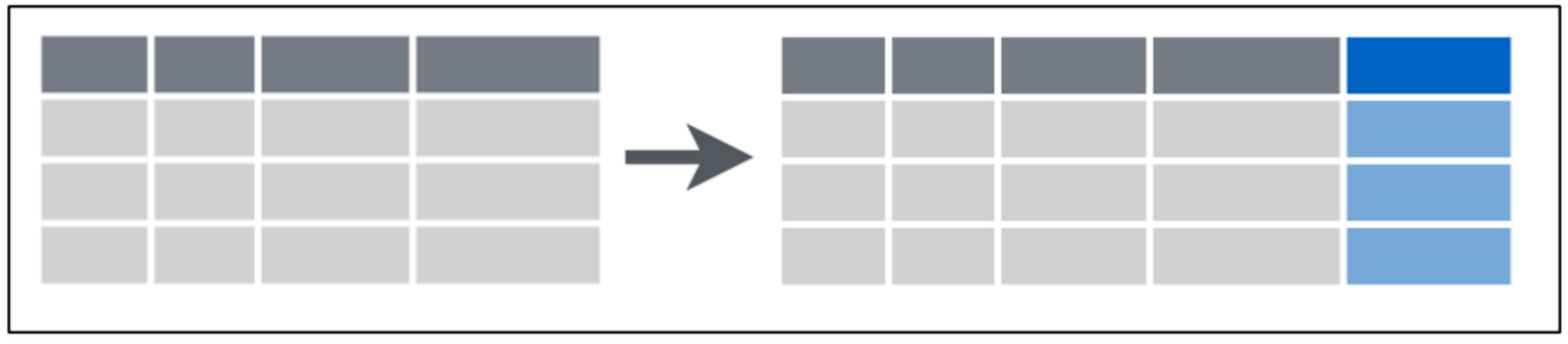
Example: Keep rows where age > 30

```
filter(data, age > 30)
```

Create columns

mutate( ) creates or changes columns.

```
mutate(data, new_column_name = expression)
```



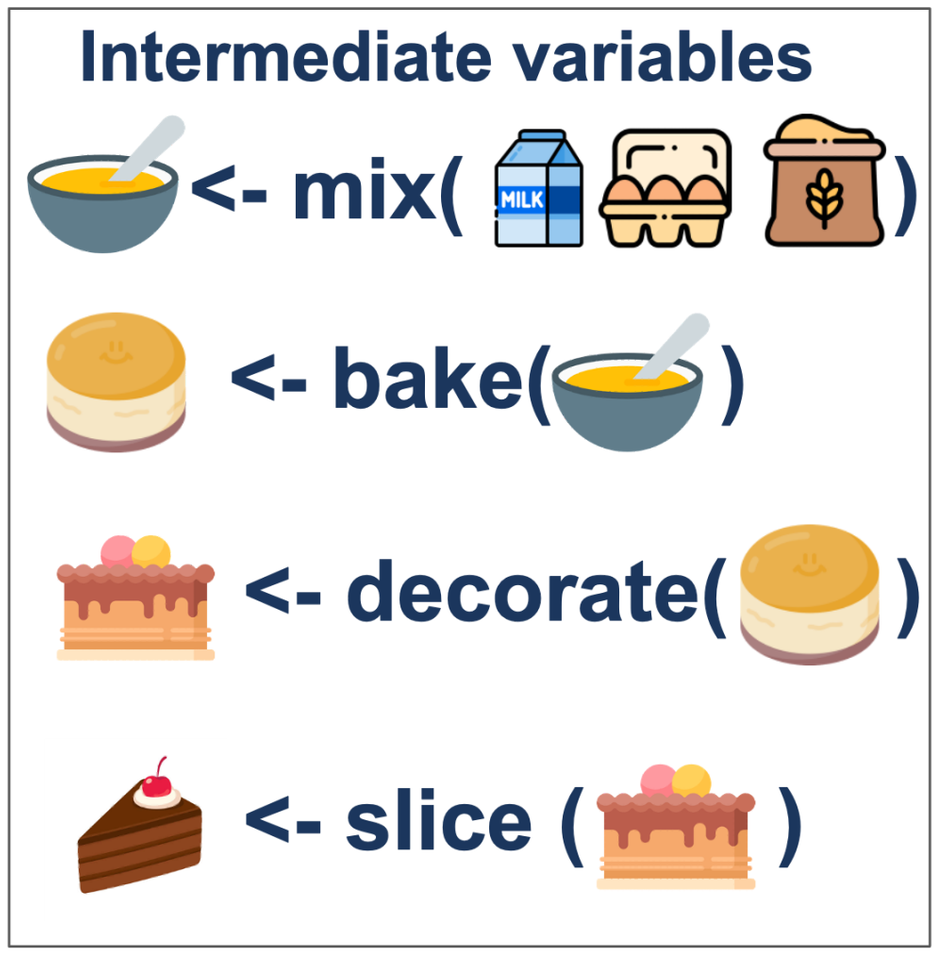
Example: Create a column called if_adult

```
data %>%
  mutate(if_adult = age >= 18)
```

The pipe operator

%>% passes a result to the next function.





Source: Arthur Welle

The pipe operator

With intermediate variables

```
data <- read_tsv("data/example_dataset_1.tsv")
selected_columns <- select(
  data, name, age, country, job
)
tidy_dataframe <- filter(selected_columns, age > 18)
```

The same steps with %>%

```
tidy_dataframe <- data %>%
  select(name, age, country, job) %>%
  filter(age > 18)
```

Comparison and logical operators

Operator	Meaning	Operator	Meaning
==	Equal to	!=	Not equal to
>	Greater than	>=	Greater than or equal to
<	Less than	<=	Less than or equal to

Logical operators combine comparison expressions.

`&` AND **`|` OR** **`!` NOT**

Combine tables

`inner_join(x, y)`	Matching keys in both tables
`left_join(x, y)`	All rows in x
`right_join(x, y)`	All rows in y
`full_join(x, y)`	All rows in x and y

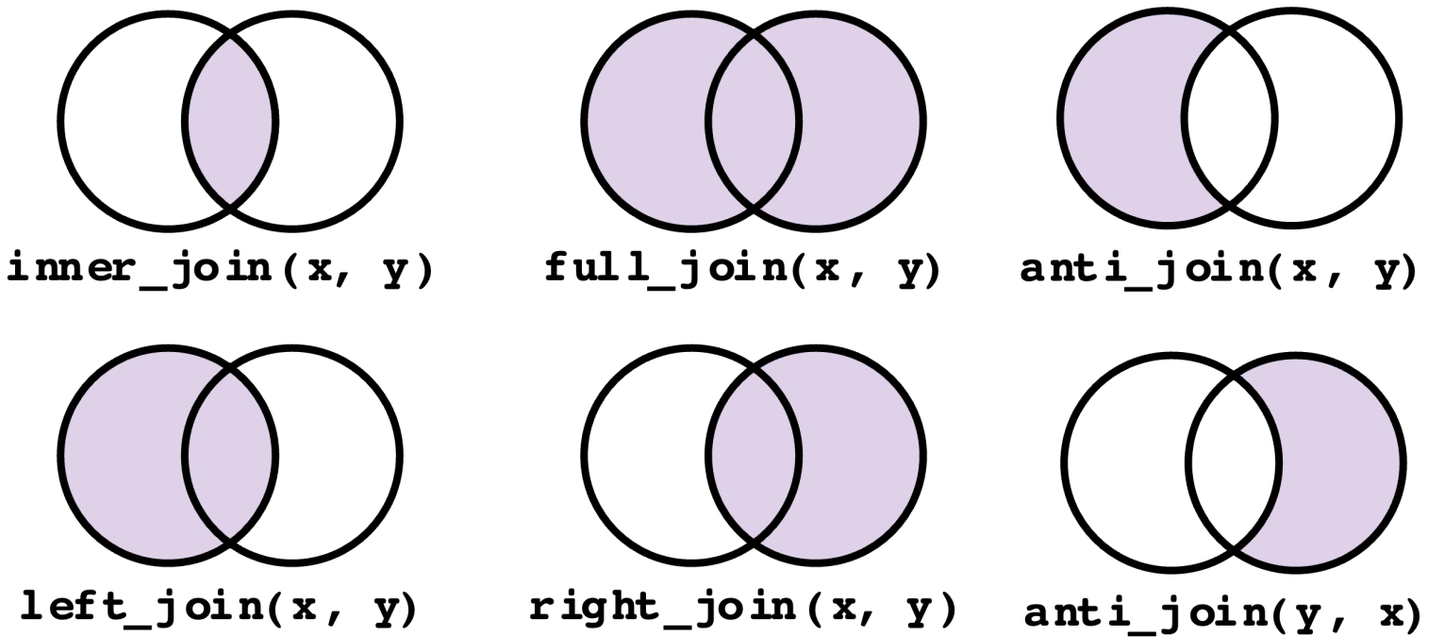
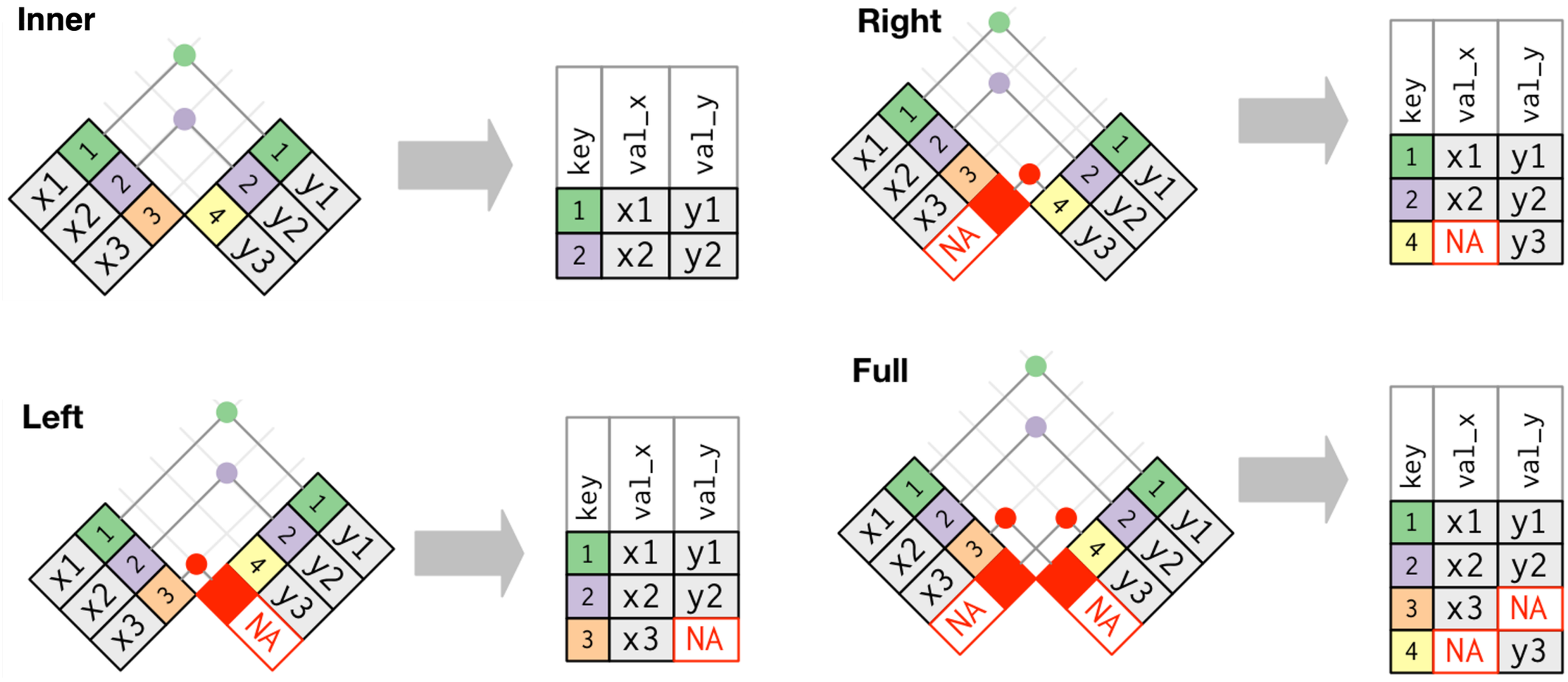


Image credit: Hugo Tavares, Introduction to the tidyverse

Combine tables




```
inner_join(x, y, by = "key")
left_join(x, y, by = "key")
```

Image credit: Hugo Tavares, Introduction to the tidyverse

Summarize data

summarize( ) calculates summary statistics.

```
data %>%
  summarize(
    min_age = min(age),
    n_samples = n()
  )
```

group_by( ) defines groups for summaries.

```
data %>%
  group_by(region) %>%
  summarize(
    mean_age = mean(age),
    se_age = sd(age) / sqrt(n()),
    .groups = "drop"
  ) %>%
  print()
```

Region	mean_age	se_age
A	25	5
B	45	6

Pivoting tables: long and wide

Long format

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

→

```
pivot_wider(data_long,
  names_from = year,
  names_prefix = "yr",
  values_from = metric)
```

Wide format

country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

←

```
pivot_longer(data_wide,
  cols = yr1960:yr2010,
  names_to = "year",
  names_prefix = "yr",
  values_to = "metric")
```

Adapted from Hugo Tavares, Introduction to the tidyverse

Data transformation with dplyr : : **CHEATSHEET**



dplyr functions work with pipes and expect **tidy data**. In tidy data:



Each **variable** is in its own **column**

&



Each **observation**, or **case**, is in its own **row**

pipes

x |> f(y) becomes **f(x, y)**

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



summarize(.data, …**)** Compute table of summaries.
mtcars |> summarize(avg = mean(mpg))



count(.data, …, wt = NULL, sort = FALSE, name = NULL**)** Count number of rows in each group defined by the variables in … Also **tally()**, **add_count()**, **add_tally()**.
mtcars |> count(cyl)

Group Cases

Use **group_by(**.data, …, .add = FALSE, .drop = TRUE**)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



```
mtcars |>
  group_by(cyl) |>
  summarize(avg = mean(mpg))
```

Use **rowwise(**.data, …**)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidyr cheat sheet for list-column workflow.



```
starwars |>
  rowwise() |>
  mutate(film_count = length(films))
```

ungroup(x, …**)** Returns ungrouped copy of table.

```
g_mtcars <- mtcars |> group_by(cyl)
ungroup(g_mtcars)
```

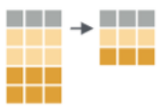
Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.

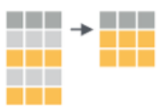


filter(.data, …, .preserve = FALSE**)** Extract rows that meet logical criteria.
mtcars |> filter(mpg > 20)

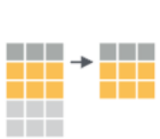


distinct(.data, …, .keep_all = FALSE**)** Remove rows with duplicate values.
mtcars |> distinct(gear)

slice(.data, …, .preserve = FALSE**)** Select rows by position.
mtcars |> slice(10:15)



slice_sample(.data, …, n, prop, weight_by = NULL, replace = FALSE**)** Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
mtcars |> slice_sample(n = 5, replace = TRUE)



slice_min(.data, order_by, …, n, prop, with_ties = TRUE**)** and **slice_max()** Select rows with the lowest and highest values.
mtcars |> slice_min(mpg, prop = 0.25)

slice_head(.data, …, n, prop**)** and **slice_tail()** Select the first or last rows.
mtcars |> slice_head(n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%	|	xor()
!=	>	>=	!is.na()	!	&	

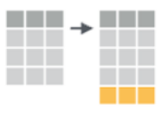
See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES



arrange(.data, …, .by_group = FALSE**)** Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
mtcars |> arrange(mpg)
mtcars |> arrange(desc(mpg))

ADD CASES



add_row(.data, …, .before = NULL, .after = NULL**)** Add one or more rows to a table.
cars |> add_row(speed = 1, dist = 1)

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



pull(.data, var = -1, name = NULL, …**)** Extract column values as a vector, by name or index.
mtcars |> pull(wt)



select(.data, …**)** Extract columns as a table.
mtcars |> select(mpg, wt)



relocate(.data, …, .before = NULL, .after = NULL**)** Move columns to new position.
mtcars |> relocate(mpg, cyl, .after = last_col())

Use these helpers with select() and across()
e.g. mtcars |> select(mpg:cyl)

contains(match**)**	**num_range(**prefix, range**)**	**:**, e.g., mpg:cyl
ends_with(match**)**	**all_of(**x**)/any_of(**x, …, vars**)**	**!**, e.g., !gear
starts_with(match**)**	**matches(**match**)**	**everything()**

MANIPULATE MULTIPLE VARIABLES AT ONCE

```
df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))
```



across(.cols, .funs, …, .names = NULL**)** Summarize or mutate multiple columns in the same way.
df |> summarize(across(everything(), mean))



c_across(.cols**)** Compute across columns in row-wise data.

```
df |>
  rowwise() |>
  mutate(x_total = sum(c_across(1:2)))
```

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function



mutate(.data, …, .keep = "all", .before = NULL, .after = NULL**)** Compute new column(s). Also **add_column()**.
mtcars |> mutate(gpm = 1 / mpg)
mtcars |> mutate(gpm = 1 / mpg, .keep = "none")



rename(.data, …**)** Rename columns. Use **rename_with()** to rename with a function.
mtcars |> rename(miles_per_gallon = mpg)

CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at **dplyr.tidyverse.org** • HTML cheatsheets at **pos.it/cheatsheets** • dplyr 1.1.4 • Updated: 2024-05

Data visualization with ggplot2

ggplot2 adds plot components using +.

```
ggplot(data) +
  aes(x = x_axis, y = y_axis) + geom_point()
```

`ggplot(data)`

Data to plot, usually a tibble

`aes()`

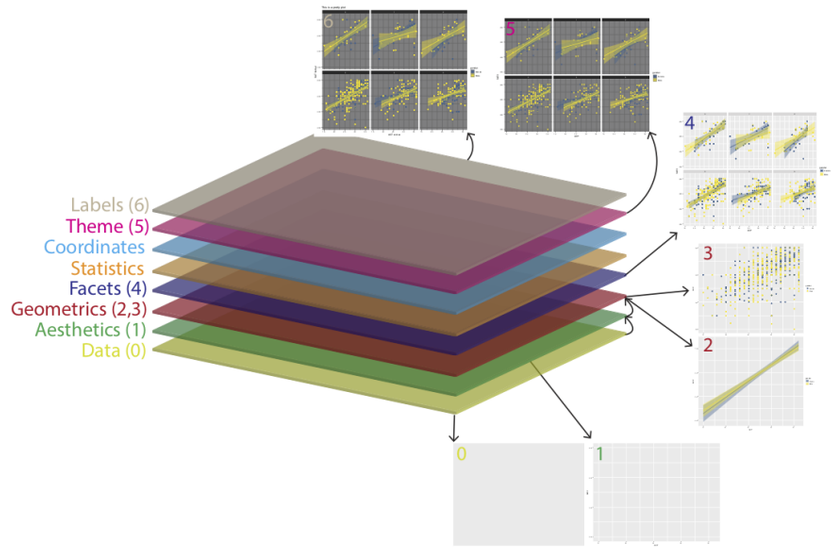
Variables mapped to the plot

`geom_*()`

Plot type, such as points or lines

`Layers`

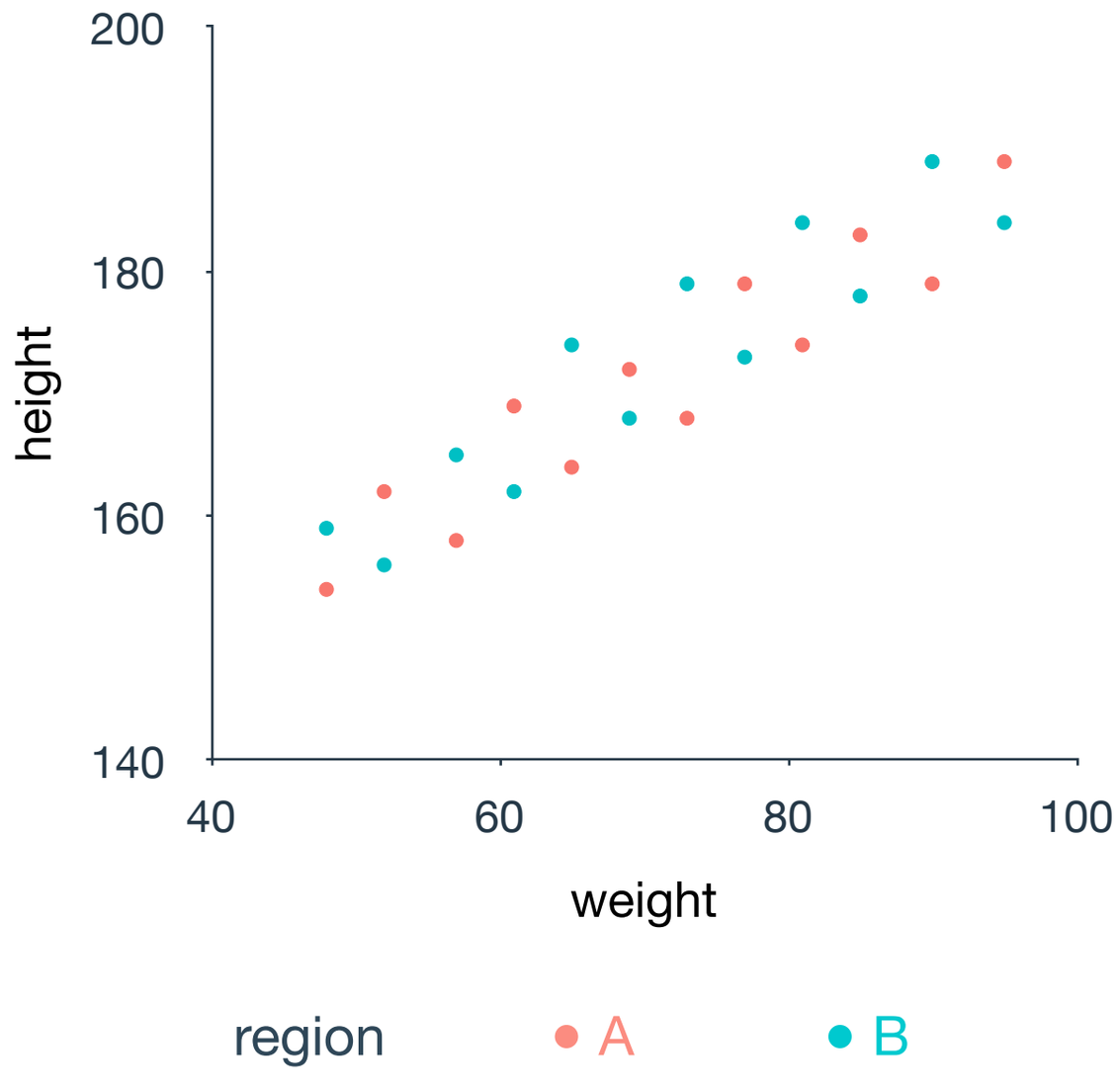
Labels, scales, themes, and more



Scatter plot of weight and height

Points colored by region

```
ggplot(data,
       aes(x = weight,
           y = height)) +
  geom_point(
    aes(color = region)) +
  labs(x = "weight",
       y = "height",
       color = "region") +
  theme_classic()
```



Resources for ggplot2

ggplot2 documentation

Function reference and examples

R Graphics Cookbook

Examples and resources for plots in R

R Graph Gallery

Plot types and additional layers